OOPS ASSIGNMENT - 8

1)Write a Java program that reads a file name from the user and attempts to open and read the file. Define a method readFile() that throws a FileNotFoundException using the throws keyword. In the main method, call this method and handle the exception using a try-catch block. Display an appropriate message if the file is not found. Use a finally block to ensure a message like "File operation attempted" is print.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class ReadFileExample {

    // Method that reads a file and throws FileNotFoundException
    public static void readFile(String fileName) throws
FileNotFoundException {
        File file = new File(fileName);
        Scanner fileScanner = new Scanner(file);

        while (fileScanner.hasNextLine()) {
            String line = fileScanner.nextLine();
            System.out.println(line);
        }

        fileScanner.close();
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the file name to read: ");
        String fileName = scanner.nextLine();
```

```
        try {
            readFile(fileName); // Attempt to read the file
        } catch (FileNotFoundException e) {
            System.out.println("Error: File not found. Please check the file
name and try again.");
        } finally {
            System.out.println("File operation attempted.");
            scanner.close();
        }
    }
}
```

```
Enter the file name to read: exp1
ERROR!
Error: File not found. Please check the file name and try again.
File operation attempted.

=== Code Execution Successful ===
```

2)Write a Java program that takes user input for a student's name, roll number, and grade, and writes this information to a file named student.txt using FileWriter. Ensure the program appends the data to the file if it already exists. Handle any exceptions using try-catch and display an appropriate message if an error occurs.

Sample File Content:

Name: Aman, Roll Number: 120112, Grade: A

Name: Parul, Roll Number: 120131, Grade: B

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class StudentInfoToFile {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Get student details from the user
        System.out.print("Enter student's name: ");
        String name = scanner.nextLine();

        System.out.print("Enter roll number: ");
        String rollNumber = scanner.nextLine();

        System.out.print("Enter grade: ");
        String grade = scanner.nextLine();

        // File to write data
        String fileName = "student.txt";

        try {
            // Create a FileWriter in append mode
            FileWriter writer = new FileWriter(fileName, true);

            // Write the student details to the file
             writer.write("Name: " + name + ", Roll Number: " + rollNumber +
", Grade: " + grade + "\n");
```

```
Enter student's name: nidhi
Enter roll number: R2142331717
Enter grade: 12th
An error occurred while writing to the file.
java.io.FileNotFoundException: student.txt (Permission denied)
    at java.base/java.io.FileOutputStream.open0(Native Method)
    at java.base/java.io.FileOutputStream.open(FileOutputStream.java:289)
    at java.base/java.io.FileOutputStream.<init>(FileOutputStream.java:230)
    at java.base/java.io.FileOutputStream.<init>(FileOutputStream.java:150)
    at java.base/java.io.FileWriter.<init>(FileWriter.java:84)
    at StudentInfoToFile.main(Main.java:25)
ERROR!
    at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke
        (DirectMethodHandleAccessor.java:103)
    at java.base/java.lang.reflect.Method.invoke(Method.java:580)
    at jdk.compiler/com.sun.tools.javac.launcher.Main.execute(Main.java:484)
ERROR!
    at jdk.compiler/com.sun.tools.javac.launcher.Main.run(Main.java:208)
    at jdk.compiler/com.sun.tools.javac.launcher.Main.main(Main.java:135)

=== Code Execution Successful ===
```

```
            // Close the writer
            writer.close();
             System.out.println("Student information successfully written to " +
fileName);

        } catch (IOException e) {
            // Handle any IOExceptions (e.g., file not found, permission issues)
            System.out.println("An error occurred while writing to the file.");
            e.printStackTrace();
        } finally {
            // Close the scanner
            scanner.close();
        }
    }
}
```

3)Write a Java program that reads the contents of a file named student.txt using FileReader and displays the data on the console. Handle FileNotFoundException if the file does not exist and display an appropriate error message. Use a try-catch block for exception handling.

```
import java.io.FileReader;
import java.io.IOException;
import java.io.BufferedReader;
import java.io.FileNotFoundException;

public class ReadFileExample {

    public static void main(String[] args) {
        String fileName = "student.txt";  // Name of the file to read

        try {
            // Create a FileReader and BufferedReader to read the file
            FileReader fileReader = new FileReader(fileName);
                                    BufferedReader bufferedReader = new
BufferedReader(fileReader);
```

```
            String line;
            // Read the file line by line and print each line
            while ((line = bufferedReader.readLine()) != null) {
                System.out.println(line);
            }

            // Close the reader
            bufferedReader.close();

        } catch (FileNotFoundException e) {
                System.out.println("Error: The file " + fileName + " was not found.");
        } catch (IOException e) {
            System.out.println("An error occurred while reading the file.");
            e.printStackTrace();
        } finally {
            System.out.println("File reading attempt completed.");
        }
    }
}
```

```
ERROR!
Error: The file student.txt was not found.
File reading attempt completed.

=== Code Execution Successful ===
```

4)Write a program that prompts the user for a text file name, opens the file using a FileInputStream (or FileReader), and counts the total number of

words and characters (excluding whitespace). Print these counts to the console. Test your program on files with varied content and edge cases (e.g., empty file, file with only whitespace, etc.)import java.io.*;
import java.util.Scanner;

public class WordCharacterCount {

    public static void main(String[] args) {
        // Create a Scanner to read user input
        Scanner scanner = new Scanner(System.in);

        // Prompt the user for the file name
        System.out.print("Enter the text file name: ");
        String fileName = scanner.nextLine();

        // Initialize counters for words and characters
        int wordCount = 0;
        int charCount = 0;

                    try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
            String line;

            // Read the file line by line
            while ((line = reader.readLine()) != null) {
                // Remove leading/trailing spaces and count characters excluding whitespace
                String[] words = line.trim().split("\\s+");
                wordCount += words.length;

                // Count characters (excluding whitespace)
                for (char c : line.toCharArray()) {
                    if (!Character.isWhitespace(c)) {
                        charCount++;
                    }
                }
            }

            // Output the results
            System.out.println("Total words: " + wordCount);

```
            System.out.println("Total characters (excluding whitespace): " +
charCount);

        } catch (IOException e) {
            System.out.println("Error reading the file: " + e.getMessage());
        } finally {
            scanner.close();
        }
    }
}
```

```
Enter the text file name: exp2
Error reading the file: exp2 (No such file or directory)

=== Code Execution Successful ===
```

5)Write a class Person with fields like name and age, implementing Serializable. In your main method, create an instance of Person and serialize it to a file (person.txt) using ObjectOutputStream. Then, read it

```
ERROR!
error: can't find main(String[]) method in class: Person

=== Code Exited With Errors ===
```

back using ObjectInputStream and confirm that the deserialized object has the same field values.

```
import java.io.*;

class Person implements Serializable {
    private String name;
    private int age;

    // Constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Getter methods
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    // Method to display the object's details
    public void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class SerializePerson {

    public static void main(String[] args) {
        // Create a Person object
        Person person = new Person("John Doe", 30);
```

```
        // Serialize the object to a file
            try (ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("person.txt"))) {
            out.writeObject(person);
            System.out.println("Object serialized successfully!");
        } catch (IOException e) {
            System.out.println("Error during serialization: " + e.getMessage());
        }

        // Deserialize the object from the file
                try (ObjectInputStream in = new ObjectInputStream(new
FileInputStream("person.txt"))) {
            Person deserializedPerson = (Person) in.readObject();
            System.out.println("Object deserialized successfully!");
            deserializedPerson.display();  // Display the deserialized object
        } catch (IOException | ClassNotFoundException e) {
                        System.out.println("Error during deserialization: " +
e.getMessage());
        }
    }
}
```